R Programming

Unit 4 - Assignment

Data Manipulation, Visualization & Machine Learning

Subject	R Programming
Unit	Unit 4 - Data Manipulation in R
Assignment	Practical Assignment
Name	Pawar Shubham Dilip
MIS No	752562026

Dataset Creation - student_data.csv

First, create the CSV file. You can do this directly from R or create it manually in Excel/Notepad. Below is the R code to create the file programmatically:

```
# Create the student dataset and save as CSV
student_data <- data.frame(
  Name       = c("Arjun", "Priya", "Rahul", "Sneha", "Karan",
                 "Meera", "Aditya", "Pooja", "Vikram", "Tanya"),
  Math       = c(85, 92, NA, 78, 65, 88, 72, NA, 90, 55),
  Science    = c(78, 85, 90, NA, 70, 82, 65, 88, 74, 60),
  English    = c(70, 88, 75, 80, NA, 76, 68, 92, 70, 72),
  Attendance = c(90, 95, 85, 88, 72, 93, 80, 91, 87, 65),
  Gender     = c("M", "F", "M", "F", "M", "F", "M", "F", "M", "F")
)

# Save to CSV
write.csv(student_data, "student_data.csv", row.names = FALSE)
cat("CSV file created: student_data.csv\n")
```

```
R 4.5.3 · ~/
> # Create the student dataset and save as CSV
> student_data <- data.frame(
+   Name       = c("Arjun", "Priya", "Rahul", "Sneha", "Karan",
+                  "Meera", "Aditya", "Pooja", "Vikram", "Tanya"),
+   Math       = c(85, 92, NA, 78, 65, 88, 72, NA, 90, 55),
+   Science    = c(78, 85, 90, NA, 70, 82, 65, 88, 74, 60),
+   English    = c(70, 88, 75, 80, NA, 76, 68, 92, 70, 72),
+   Attendance = c(90, 95, 85, 88, 72, 93, 80, 91, 87, 65),
+   Gender     = c("M", "F", "M", "F", "M", "F", "M", "F", "M", "F")
+ )
>
> # Save to CSV
> write.csv(student_data, "student_data.csv", row.names = FALSE)
> cat("CSV file created: student_data.csv\n")
CSV file created: student_data.csv
>
```

Section 1: Importing & Reading Data

```
# 1. Import the CSV file
df <- read.csv("student_data.csv", stringsAsFactors = FALSE)

# 2. Display the dataset
print(df)

# 3. Check structure - column names, data types, sample values
str(df)

# 4. First 5 and last 5 rows
head(df, 5)
tail(df, 5)
```

R 4.5.3 · ~/

```
> # 1. Import the CSV file
> df <- read.csv("student_data.csv", stringsAsFactors = FALSE)
>
> # 2. Display the dataset
> print(df)
     Name Math Science English Attendance Gender
1   Arjun   85      78      70         90      M
2   Priya   92      85      88         95      F
3   Rahul   NA      90      75         85      M
4   Sneha   78      NA      80         88      F
5   Karan   65      70      NA         72      M
6   Meera   88      82      76         93      F
7  Aditya   72      65      68         80      M
8   Pooja   NA      88      92         91      F
9  Vikram   90      74      70         87      M
10  Tanya   55      60      72         65      F
>
> # 3. Check structure - column names, data types, sample values
> str(df)
'data.frame':   10 obs. of  6 variables:
 $ Name      : chr  "Arjun" "Priya" "Rahul" "Sneha" ...
 $ Math      : int  85 92 NA 78 65 88 72 NA 90 55
 $ Science   : int  78 85 90 NA 70 82 65 88 74 60
 $ English   : int  70 88 75 80 NA 76 68 92 70 72
 $ Attendance: int  90 95 85 88 72 93 80 91 87 65
 $ Gender    : chr  "M" "F" "M" "F" ...
>
> # 4. First 5 and last 5 rows
> head(df, 5)
   Name Math Science English Attendance Gender
1 Arjun   85      78      70         90      M
2 Priya   92      85      88         95      F
3 Rahul   NA      90      75         85      M
4 Sneha   78      NA      80         88      F
5 Karan   65      70      NA         72      M
> tail(df, 5)
     Name Math Science English Attendance Gender
6   Meera   88      82      76         93      F
7  Aditya   72      65      68         80      M
8   Pooja   NA      88      92         91      F
9  Vikram   90      74      70         87      M
10  Tanya   55      60      72         65      F
```

Section 2: Data Cleaning

```
# 5. Identify missing values (TRUE = NA at that position)
is.na(df)

# 6. Count total missing values per column
colSums(is.na(df))

# 7. Replace NA in numeric columns with column mean
df$Math    <- ifelse(is.na(df$Math),    round(mean(df$Math,    na.rm=TRUE)),
df$Math)
df$Science <- ifelse(is.na(df$Science), round(mean(df$Science, na.rm=TRUE)),
df$Science)
df$English <- ifelse(is.na(df$English), round(mean(df$English, na.rm=TRUE)),
df$English)

# 8. Verify - should show 0 NAs in all numeric columns
colSums(is.na(df))

cat("Dataset is clean - no missing values.\n")
```

```
R 4.5.3 · ~/
>
> # 5. Identify missing values (TRUE = NA at that position)
> is.na(df)
       Name  Math Science English Attendance Gender
 [1,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
 [2,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
 [3,] FALSE  TRUE   FALSE   FALSE      FALSE  FALSE
 [4,] FALSE FALSE    TRUE   FALSE      FALSE  FALSE
 [5,] FALSE FALSE   FALSE    TRUE      FALSE  FALSE
 [6,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
 [7,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
 [8,] FALSE  TRUE   FALSE   FALSE      FALSE  FALSE
 [9,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
[10,] FALSE FALSE   FALSE   FALSE      FALSE  FALSE
>
> # 6. Count total missing values per column
> colSums(is.na(df))
      Name       Math    Science    English Attendance     Gender
         0          2          1          1          0          0
>
> # 7. Replace NA in numeric columns with column mean
> df$Math    <- ifelse(is.na(df$Math),    round(mean(df$Math,    na.rm=TRUE)), df$Math)
> df$Science <- ifelse(is.na(df$Science), round(mean(df$Science, na.rm=TRUE)), df$Science)
> df$English <- ifelse(is.na(df$English), round(mean(df$English, na.rm=TRUE)), df$English)
>
> # 8. Verify - should show 0 NAs in all numeric columns
> colSums(is.na(df))
      Name       Math    Science    English Attendance     Gender
         0          0          0          0          0          0
>
> cat("Dataset is clean - no missing values.\n")
Dataset is clean - no missing values.
>
```

Section 3: Data Manipulation using dplyr

```
# 9. Install and load dplyr
install.packages("dplyr")
library(dplyr)

# 10. Create Average column
df <- df %>% mutate(Average = round((Math + Science + English) / 3, 2))

# 11. Filter students with Average > 75
high_performers <- df %>% filter(Average > 75)
print(high_performers)

# 12. Select only Name and Average
df %>% select(Name, Average)

# 13. Arrange in descending order of Average
df %>% arrange(desc(Average))

# 14. Group by Gender and find average marks
df %>%
  group_by(Gender) %>%
  summarise( Avg_Math    = round(mean(Math), 2),
             Avg_Science = round(mean(Science), 2),
             Avg_English = round(mean(English), 2),
             Avg_Overall = round(mean(Average), 2),
             Count       = n())
```

```
R 4.5.3 · ~/
> # 10. Create Average column
> df <- df %>%
+   mutate(Average = round((Math + Science + English) / 3, 2))
>
> # 11. Filter students with Average > 75
> high_performers <- df %>% filter(Average > 75)
> print(high_performers)
    Name Math Science English Attendance Gender Average
1  Arjun   85      78      70         90      M   77.67
2  Priya   92      85      88         95      F   88.33
3  Rahul   78      90      75         85      M   81.00
4  Sneha   78      77      80         88      F   78.33
5  Meera   88      82      76         93      F   82.00
6  Pooja   78      88      92         91      F   86.00
7 Vikram   90      74      70         87      M   78.00
>
> # 12. Select only Name and Average
> df %>% select(Name, Average)
     Name Average
1   Arjun   77.67
2   Priya   88.33
3   Rahul   81.00
4   Sneha   78.33
5   Karan   70.67
6   Meera   82.00
7  Aditya   68.33
8   Pooja   86.00
9  Vikram   78.00
10  Tanya   62.33
>
> # 13. Arrange in descending order of Average
> df %>% arrange(desc(Average))
     Name Math Science English Attendance Gender Average
1   Priya   92      85      88         95      F   88.33
2   Pooja   78      88      92         91      F   86.00
3   Meera   88      82      76         93      F   82.00
4   Rahul   78      90      75         85      M   81.00
5   Sneha   78      77      80         88      F   78.33
6  Vikram   90      74      70         87      M   78.00
7   Arjun   85      78      70         90      M   77.67
8   Karan   65      70      77         72      M   70.67
9  Aditya   72      65      68         80      M   68.33
```

```
> # 14. Group by Gender and find average marks
> df %>%
+   group_by(Gender) %>%
+   summarise(
+     Avg_Math    = round(mean(Math), 2),
+     Avg_Science = round(mean(Science), 2),
+     Avg_English = round(mean(English), 2),
+     Avg_Overall = round(mean(Average), 2),
+     Count       = n()
+   )
# A tibble: 2 × 6
  Gender Avg_Math Avg_Science Avg_English Avg_Overall Count
  <chr>     <dbl>       <dbl>       <dbl>       <dbl> <int>
1 F          78.2        78.4        81.6        79.4     5
2 M          78          75.4        72          75.1     5
>
```

Section 4: Exploratory Data Analysis (EDA)

```
# 15. Summary statistics
summary(df)

# 16. Mean of each subject
cat("Mean Math    :", round(mean(df$Math), 2), "\n")
cat("Mean Science :", round(mean(df$Science), 2), "\n")
cat("Mean English :", round(mean(df$English), 2), "\n")
```

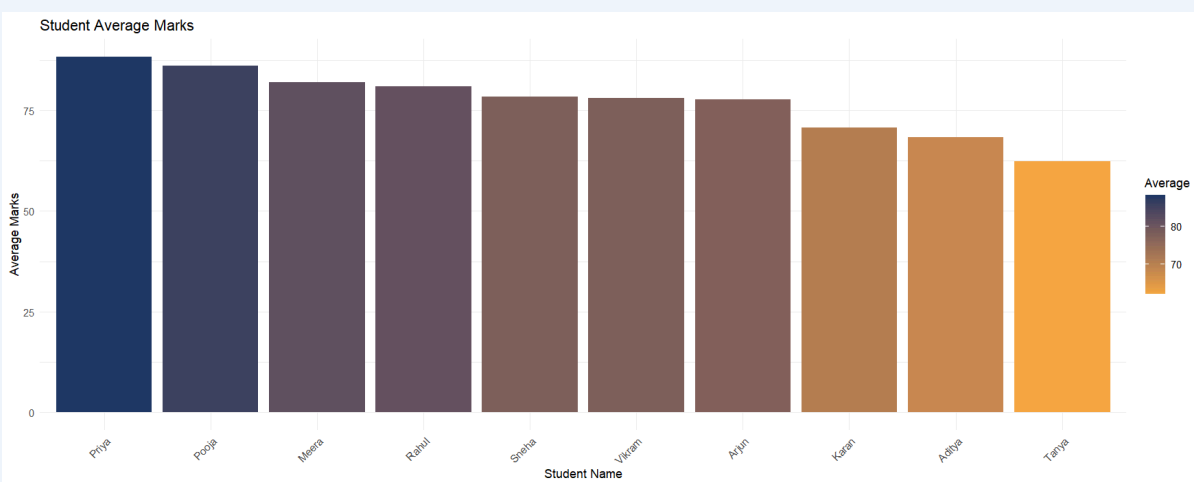
```r
# 17. Correlation between subjects
cor_matrix <- cor(df[ , c("Math", "Science", "English")])
print(round(cor_matrix, 3))

# 18. Subject with highest average
subject_means <- c(Math = mean(df$Math), Science = mean(df$Science), English =
mean(df$English))
cat("Subject with highest average:", names(which.max(subject_means)), "\n")
cat("Score:", round(max(subject_means), 2), "\n")
```

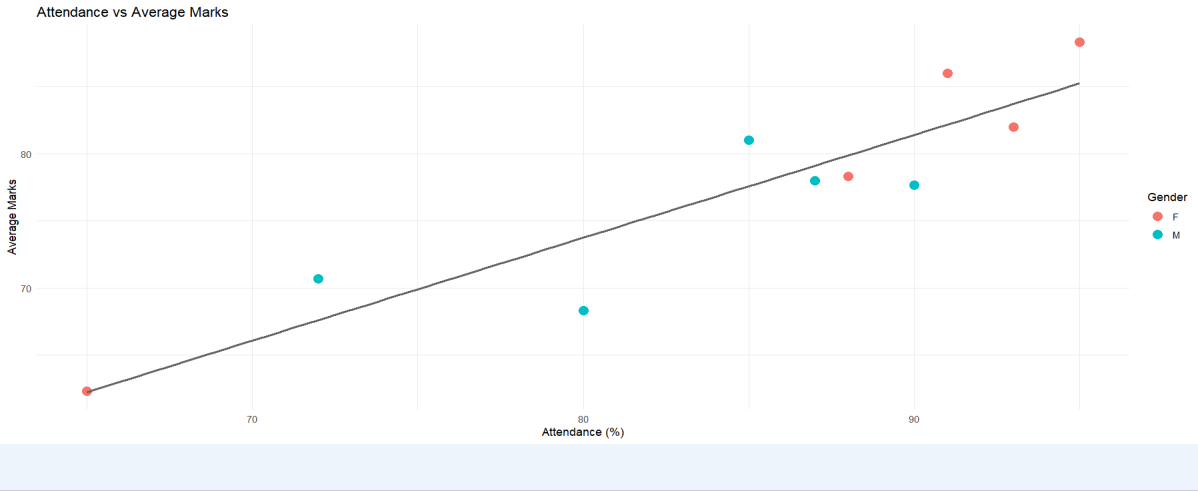
R 4.5.3 · ~/

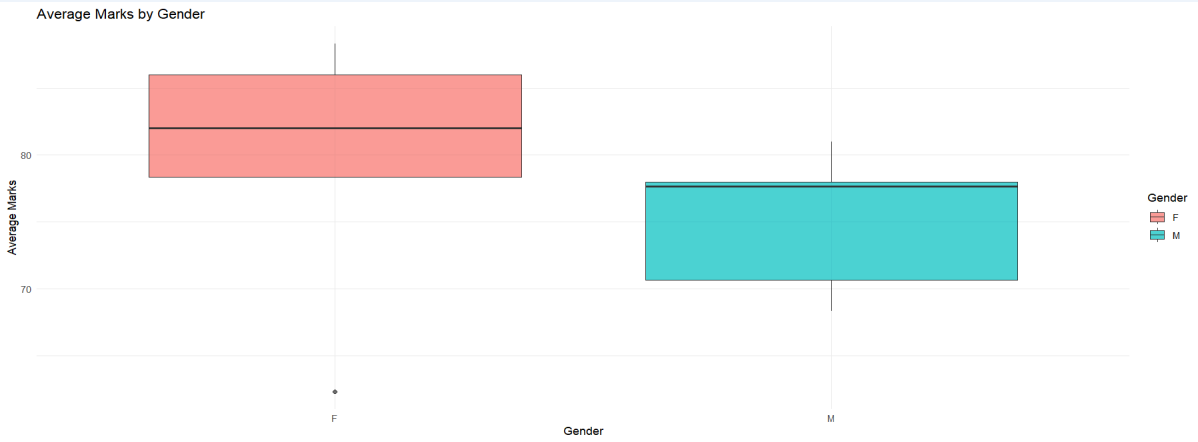
```
2 M          78          75.4        72          75.1     5
> # 15. Summary statistics
> summary(df)
     Name                Math          Science         English        Attendance
 Length:10          Min.   :55.00   Min.   :60.00   Min.   :68.00   Min.   :65.00
 Class :character   1st Qu.:73.50   1st Qu.:71.00   1st Qu.:70.50   1st Qu.:81.25
 Mode  :character   Median :78.00   Median :77.50   Median :75.50   Median :87.50
                    Mean   :78.10   Mean   :76.90   Mean   :76.80   Mean   :84.60
                    3rd Qu.:87.25   3rd Qu.:84.25   3rd Qu.:79.25   3rd Qu.:90.75
                    Max.   :92.00   Max.   :90.00   Max.   :92.00   Max.   :95.00
    Gender             Average
 Length:10          Min.   :62.33
 Class :character   1st Qu.:72.42
 Mode  :character   Median :78.17
                    Mean   :77.27
                    3rd Qu.:81.75
                    Max.   :88.33
>
> # 16. Mean of each subject
> cat("Mean Math    :", round(mean(df$Math), 2), "\n")
Mean Math    : 78.1
> cat("Mean Science :", round(mean(df$Science), 2), "\n")
Mean Science : 76.9
> cat("Mean English :", round(mean(df$English), 2), "\n")
Mean English : 76.8
>
> # 17. Correlation between subjects
> cor_matrix <- cor(df[ , c("Math", "Science", "English")])
> print(round(cor_matrix, 3))
         Math Science English
Math    1.000   0.666   0.217
Science 0.666   1.000   0.612
English 0.217   0.612   1.000
>
> # 18. Subject with highest average
> subject_means <- c(Math = mean(df$Math), Science = mean(df$Science), English = mean(df$English))
> cat("Subject with highest average:", names(which.max(subject_means)), "\n")
Subject with highest average: Math
> cat("Score:", round(max(subject_means), 2), "\n")
Score: 78.1
>
```

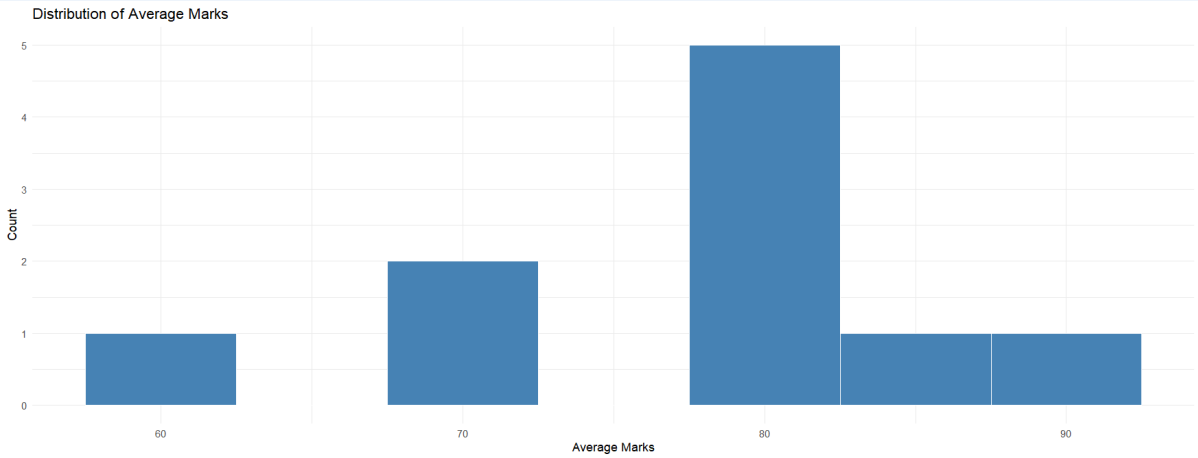
Section 5: Data Visualization using ggplot2

```
# 19. Install and load ggplot2
install.packages("ggplot2")   # run once
library(ggplot2)

# 20. Bar plot: Student vs Average marks
ggplot(df, aes(x = reorder(Name, -Average), y = Average, fill = Average)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "#f5a742", high = "#1F3864") +
  labs(title = "Student Average Marks", x = "Student Name", y = "Average Marks") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))

# 21. Scatter plot: Attendance vs Average
ggplot(df, aes(x = Attendance, y = Average, color = Gender, label = Name)) +
  geom_point(size = 4) +
  geom_smooth(method = "lm", se = FALSE, color = "gray40") +
  labs(title = "Attendance vs Average Marks", x = "Attendance (%)", y = "Average
Marks") +
  theme_minimal()

# 22. Boxplot: Average marks by Gender
ggplot(df, aes(x = Gender, y = Average, fill = Gender)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = "Average Marks by Gender", x = "Gender", y = "Average Marks") +
  theme_minimal()

# 23. Histogram: Distribution of Average marks
ggplot(df, aes(x = Average)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white") +
  labs(title = "Distribution of Average Marks", x = "Average Marks", y = "Count") +
  theme_minimal()
```




Attendance vs Average Marks
Average Marks
80
70
70
80
90
Attendance (%)
Gender
F
M


Average Marks by Gender
Average Marks
80
70
F
M
Gender
Gender
F
M


Distribution of Average Marks
Count
5
4
3
2
1
0
60
70
80
90
Average Marks

Section 6: Statistics with R

```
# 24. Mean of Average marks
mean_avg <- mean(df$Average)
cat("Mean of Average marks:", round(mean_avg, 2), "\n")

# 25. Median of Average marks
median_avg <- median(df$Average)
cat("Median of Average marks:", round(median_avg, 2), "\n")

# 26. Standard deviation
sd_avg <- sd(df$Average)
cat("Standard Deviation:", round(sd_avg, 2), "\n")

# 27. Observe and compare mean vs median
cat("\n--- Mean vs Median Comparison ---\n")
cat("Mean   :", round(mean_avg, 2), "\n")
cat("Median:", round(median_avg, 2), "\n")
if (abs(mean_avg - median_avg) < 2) {
  cat("Observation: Mean and Median are close - distribution is approximately
symmetric.\n")
} else if (mean_avg > median_avg) {
  cat("Observation: Mean > Median - distribution is right-skewed (high scores pull
up mean).\n")
} else {
  cat("Observation: Mean < Median - distribution is left-skewed.\n")
}
```

```
R 4.5.3 · ~/
> # 24. Mean of Average marks
> mean_avg <- mean(df$Average)
> cat("Mean of Average marks:", round(mean_avg, 2), "\n")
Mean of Average marks: 77.27
>
> # 25. Median of Average marks
> median_avg <- median(df$Average)
> cat("Median of Average marks:", round(median_avg, 2), "\n")
Median of Average marks: 78.16
>
> # 26. Standard deviation
> sd_avg <- sd(df$Average)
> cat("Standard Deviation:", round(sd_avg, 2), "\n")
Standard Deviation: 8.05
>
> # 27. Observe and compare mean vs median
> cat("\n--- Mean vs Median Comparison ---\n")

--- Mean vs Median Comparison ---
> cat("Mean   :", round(mean_avg, 2), "\n")
Mean   : 77.27
> cat("Median:", round(median_avg, 2), "\n")
Median: 78.16
> if (abs(mean_avg - median_avg) < 2) {
+   cat("Observation: Mean and Median are close - distribution is approximately symmetric.\n")
+ } else if (mean_avg > median_avg) {
+   cat("Observation: Mean > Median - distribution is right-skewed (high scores pull up mean).\n")
+ } else {
+   cat("Observation: Mean < Median - distribution is left-skewed.\n")
+ }
Observation: Mean and Median are close - distribution is approximately symmetric.
```

Section 7: Machine Learning in R

Build a simple linear regression model to predict average marks from attendance. This is supervised learning - we train the model on known data and use it to predict new values.

```
# 28. Build linear regression model
# Formula: Average ~ Attendance means 'predict Average using Attendance'
model <- lm(Average ~ Attendance, data = df)

# 29. Display model summary
summary(model)
# Key outputs to look for:
# - Coefficients: Intercept and Attendance slope
# - R-squared: how much variance in Average is explained by Attendance
# - p-value: statistical significance (< 0.05 = significant)

# 30. Predict Average marks using the model
predicted_values <- predict(model, newdata = df)
cat("\nPredicted Average Marks:\n")
print(round(predicted_values, 2))

# 31. Compare actual vs predicted
comparison <- data.frame(
  Name      = df$Name,
  Actual    = df$Average,
  Predicted = round(predicted_values, 2),
  Residual  = round(df$Average - predicted_values, 2)
)
cat("\n--- Actual vs Predicted ---\n")
print(comparison)
# Residual = Actual - Predicted (positive = model under-predicted)
```

```
> # 29. Display model summary
> summary(model)

Call:
lm(formula = Average ~ Attendance, data = df)

Residuals:
    Min      1Q  Median      3Q     Max 
-5.4088 -1.6661 -0.5066  3.0834  3.8265 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  12.3954    10.1794   1.218 0.258034    
Attendance    0.7668     0.1196   6.410 0.000207 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.447 on 8 degrees of freedom
Multiple R-squared:  0.837,	Adjusted R-squared:  0.8166 
F-statistic: 41.08 on 1 and 8 DF,  p-value: 0.000207

> # Key outputs to look for:
> # - Coefficients: Intercept and Attendance slope
> # - R-squared: how much variance in Average is explained by Attendance
> # - p-value: statistical significance (< 0.05 = significant)
```

```
> # 31. Compare actual vs predicted
> comparison <- data.frame(
+   Name      = df$Name,
+   Actual    = df$Average,
+   Predicted = round(predicted_values, 2),
+   Residual  = round(df$Average - predicted_values, 2)
+ )
> cat("\n--- Actual vs Predicted ---\n")

--- Actual vs Predicted ---
> print(comparison)
     Name Actual Predicted Residual
1   Arjun  77.67     81.41    -3.74
2   Priya  88.33     85.24     3.09
3   Rahul  81.00     77.57     3.43
4   Sneha  78.33     79.87    -1.54
5   Karan  70.67     67.60     3.07
6   Meera  82.00     83.71    -1.71
7  Aditya  68.33     73.74    -5.41
8   Pooja  86.00     82.17     3.83
9  Vikram  78.00     79.11    -1.11
10  Tanya  62.33     62.24     0.09
> # Residual = Actual - Predicted (positive = model under-predicted)
```

Section 8: Exporting Data

```
# 32. Save cleaned dataset as CSV
write.csv(df, "student_data_cleaned.csv", row.names = FALSE)
cat("Cleaned CSV saved: student_data_cleaned.csv\n")

# 33. Save dataset as R object file (.RData)
saveRDS(df, "student_data.RDS")
cat("R object saved: student_data.RDS\n")

# 34. Load the saved R object file
df_loaded <- readRDS("student_data.RDS")
cat("R object loaded successfully.\n")
identical(df, df_loaded)
head(df_loaded, 3)
```

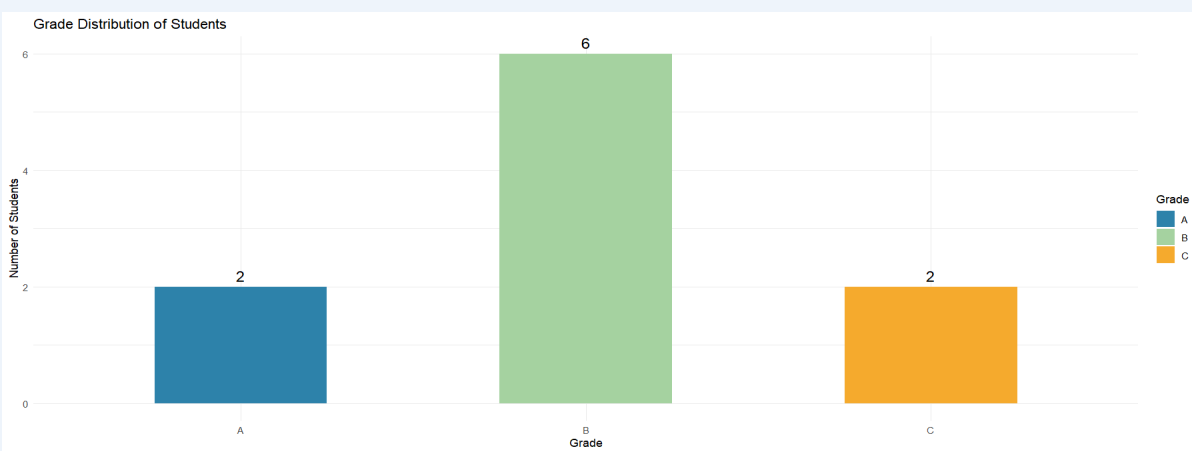
```
R 4.5.3 · ~/
> # 32. Save cleaned dataset as CSV
> write.csv(df, "student_data_cleaned.csv", row.names = FALSE)
> cat("Cleaned CSV saved: student_data_cleaned.csv\n")
Cleaned CSV saved: student_data_cleaned.csv
>
> # 33. Save dataset as R object file (.RData)
> saveRDS(df, "student_data.RDS")
> cat("R object saved: student_data.RDS\n")
R object saved: student_data.RDS
>
> # 34. Load the saved R object file
> df_loaded <- readRDS("student_data.RDS")
> cat("R object loaded successfully.\n")
R object loaded successfully.
> identical(df, df_loaded)
[1] TRUE
> head(df_loaded, 3)
   Name Math Science English Attendance Gender Average
1 Arjun   85      78      70         90      M   77.67
2 Priya   92      85      88         95      F   88.33
3 Rahul   78      90      75         85      M   81.00
```

Final Challenge (Q35-38)

```
# 35. Create Grade column based on Average
df$Grade <- ifelse(df$Average >= 85, "A",
           ifelse(df$Average >= 70, "B", "C"))

cat("\n--- Grade Distribution ---\n")
print(table(df$Grade))
print(df[ , c("Name", "Average", "Grade")])

# 36. Top 3 students by Average
top3 <- df[order(-df$Average), ][1:3, ]
cat("\n--- Top 3 Students ---\n")
print(top3[ , c("Name", "Average", "Grade")])

# 37. Which Gender performs better overall
gender_performance <- tapply(df$Average, df$Gender, mean)
cat("\n--- Gender-wise Average Performance ---\n")
print(round(gender_performance, 2))
better_gender <- names(which.max(gender_performance))
cat("Better performing gender:", better_gender, "\n")

# 38. Additional visualization: Grade distribution pie-like bar chart
grade_counts <- as.data.frame(table(df$Grade))
colnames(grade_counts) <- c("Grade", "Count")

ggplot(grade_counts, aes(x = Grade, y = Count, fill = Grade)) +
  geom_bar(stat = "identity", width = 0.5) +
  scale_fill_manual(values = c("A" = "#2E86AB", "B" = "#A8D5A2", "C" = "#F6AE2D"))
+
  geom_text(aes(label = Count), vjust = -0.5, size = 5) +
  labs(title = "Grade Distribution of Students",
       x = "Grade", y = "Number of Students") +
  theme_minimal()
```

```
R 4.5.3 · ~/
> # 35. Create Grade column based on Average
> df$Grade <- ifelse(df$Average >= 85, "A",
+                    ifelse(df$Average >= 70, "B", "C"))
>
> cat("\n--- Grade Distribution ---\n")

--- Grade Distribution ---
> print(table(df$Grade))

A B C
2 6 2
> print(df[ , c("Name", "Average", "Grade")])
     Name Average Grade
1   Arjun   77.67     B
2   Priya   88.33     A
3   Rahul   81.00     B
4   Sneha   78.33     B
5   Karan   70.67     B
6   Meera   82.00     B
7  Aditya   68.33     C
8   Pooja   86.00     A
9  Vikram   78.00     B
10  Tanya   62.33     C
>
> # 36. Top 3 students by Average
> top3 <- df[order(-df$Average), ][1:3, ]
> cat("\n--- Top 3 Students ---\n")

--- Top 3 Students ---
> print(top3[ , c("Name", "Average", "Grade")])
   Name Average Grade
2 Priya   88.33     A
8 Pooja   86.00     A
6 Meera   82.00     B
>
> # 37. Which Gender performs better overall
> gender_performance <- tapply(df$Average, df$Gender, mean)
> cat("\n--- Gender-wise Average Performance ---\n")
```

```
--- Gender-wise Average Performance ---
> print(round(gender_performance, 2))
    F     M
79.40 75.13
> better_gender <- names(which.max(gender_performance))
> cat("Better performing gender:", better_gender, "\n")
Better performing gender: F
>
```



End of Assignment

R Programming - Unit 4 | Data Manipulation, Visualization & Machine Learning